ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

RAPPORT DE MINI-PROJET : PERCEPTRON MULTICOUCHE (MLP) SOUS PYTORCH

Réaliser par : EZ-ZAHRAOUI El-Mehdi

Groupe : 5 DDSI G :2

Année scolaire : 2026 - 2027

Liste des Matière

1. Introduction et Problématique 1

1.1 Contexte et Objectifs du Mini-Projet 1

1.2 Problématique Centrale de l'Étude 1

1.3 Spécifications du Dataset Fashion-MNIST 1

2. Rappels Théoriques sur les MLP 2

2.1 Formalisme Mathématique d'un MLP à une Couche Cachée 2

2.2 Rôle Fondamental de la Fonction d'Activation Non-Linéaire (ReLU) 2

2.3 Entraînement supervisé et Fonction de Perte (Entropie Croisée & SGD) 3

2.4 Concepts de Capacité, Sous-apprentissage et Surapprentissage 3

3. Implémentation From Scratch 3

3.1 Déclaration Manuelle des Paramètres (W1, b1, W2, b2) et Rôle de nn. 3

3.2 Écriture Explicite de la Transformation Vectorielle et de la Passe Avant (forward). 4

3.3 Analyse de l'Initialisation des Poids et Gestion des Formes des Tenseurs. 4

4. Implémentation Concise 5

4.1 Construction de l'Architecture avec les Abstractions Haut Niveau (nn.Sequential, nn.Flatten, nn.Linear, nn.ReLU) 5

4.2 Tableau Comparatif : Implémentation From Scratch vs Implémentation Concise 6

4.3 Inspection Interne du Modèle (Dimensions, Poids, Biais et Mécanisme d'Autograd) 6

5. Protocole Expérimental 7

5.1 Environnement d'Exécution et Boucle d'Entraînement Générique 7

5.2 Variation de la Taille de la Couche Cachée 8

5.3 Variation du Taux d'Apprentissage 8

6. Résultats et Interprétation 9

6.1 Analyse de la Convergence du Modèle de Référence (Baseline) 9

6.2 Analyse Comparative de l'Impact de num_hiddens sur la Précision de Test 10

6.3 Diagnostic et Identification Expérimentale du Sous-apprentissage et Surapprentissage .. 11

6.4 Sélection et Justification de la Configuration Optimale 11

7. Analyse Critique et Limites 12

7.1 Limites Fondamentales du MLP en Vision par Ordinateur (Perte de la Structure Spatiale 2D et Absence d'Invariance) 12

7.2 Problématique de l'Explosion du Nombre de Paramètres sur Images Haute Définition 12

7.3 Pertinence et Comparaison avec les Réseaux Convolutifs (CNN) 13

8. Conclusion et Perspectives 13

8.1 Synthèse des Apports et des Résultats Obtenus 13

8.2 Réponses Explicites aux Questions de Synthèse.....14
8.3 Perspectives d'Amélioration (Early Stopping, Dropout, Régularisation L2) 14

Liste des Figures

Figure 1:Structure séquentielle et nombre total de paramètres du modèle concis.....5
Figure 2:Validation des dimensions des tenseurs et inspection des poids, biais et gradients. 7
Figure 3:Suivi textuel de l'entraînement et des performances par époque (Train/Test Loss & Accuracy).....9
Figure 4:Évolution de la Perte (Loss) et de la Précision (Accuracy) du modèle de référence au fil des époques. 9
Figure 5:Comparaison de la Précision de Test en fonction du nombre de neurones cachés.....10

Liste des Tables

Tableau 1:Tableau Comparatif.....6

1. Introduction et Problématique

1.1 Contexte et Objectifs du Mini-Projet

Le Perceptron Multicouche (MLP) constitue l'architecture fondamentale des réseaux de neurones artificiels profonds dans le domaine de l'apprentissage supervisé et du Deep Learning. Ce mini-projet s'intéresse à la conception, l'implémentation, l'entraînement et l'évaluation rigoureuse d'un MLP appliqué à la classification d'images. Les principaux objectifs consistent à :

- Comprendre et reproduire les mécanismes de bas niveau d'un réseau de neurones à travers une implémentation *from scratch*.
- Maîtriser l'utilisation des API de haut niveau de PyTorch via une implémentation concise.
- Mener une étude expérimentale rigoureuse pour analyser l'impact des choix architecturaux et des hyperparamètres sur la performance et la généralisation du modèle.

1.2 Problématique Centrale de l'Étude

La question scientifique au cœur de cette étude s'articule autour de l'optimisation des performances des réseaux de neurones :

- Comment concevoir, implémenter, entraîner et évaluer rigoureusement un perceptron multicouche pour une tâche de classification, tout en comprenant l'impact de ses choix architecturaux et de ses hyperparamètres sur la performance et la généralisation ?

1.3 Spécifications du Dataset Fashion-MNIST

Le modèle est évalué sur la base de données **Fashion-MNIST**, qui se compose de 70 000 images en niveaux de gris de dimensions 28 × 28 pixels, réparties équitablement en 10 classes d'articles de mode distinctes.

- **Nombre total d'exemples :** 70 000 images
- **Taille des images :** 28 × 28 pixels (1 canal, niveaux de gris)
- **Dimension d'entrée vectorisée (d) :** 784 caractéristiques (28 × 28)
- **Nombre de classes (c) :** 10 catégories d'articles de mode
- **Répartition Train / Test :** 60 000 exemples d'entraînement / 10 000 exemples de test
- **Format des Tenseurs PyTorch :** [Batch_Size, 1, 28, 28] => [Batch_Size, 784]

2. Rappels Théoriques sur les MLP

2.1 Formalisme Mathématique d'un MLP à une Couche Cachée

Le Perceptron Multicouche (MLP) est une architecture de réseau de neurones artificiels constituée d'une ou plusieurs couches cachées situées entre la couche d'entrée et la couche de sortie. Pour un lot d'images vectorisées en entrée, le modèle calcule les activations de la couche cachée en effectuant une transformation affine des données, suivie d'une fonction d'activation non linéaire. Les résultats de la couche cachée sont ensuite transmis à la couche de sortie pour calculer les scores de prédiction (logits).

Couche cachée :

- Reçoit en entrée le vecteur des caractéristiques de l'image (784 entrées pour une image 28x28).
- Effectue la multiplication avec la première matrice de poids (W1) et ajoute le premier vecteur de biais (b1).
- Applique la fonction d'activation non linéaire (ReLU) pour produire le tenseur des activations cachées (H).

Couche de sortie :

- Prend les activations (H) en entrée.
- Effectue la multiplication avec la seconde matrice de poids (W2) et ajoute le second vecteur de biais (b2).
- Génère les 10 prédictions finales correspondant aux 10 classes du dataset Fashion-MNIST.

2.2 Rôle Fondamental de la Fonction d'Activation Non-Linéaire (ReLU)

La fonction d'activation ReLU (Rectified Linear Unit) transforme les valeurs d'entrée en conservant uniquement les valeurs positives et en ramenant toutes les valeurs négatives à zéro. Son rôle est déterminant dans l'architecture :

- **Modélisation de la complexité :** Elle introduit de la non-linéarité au sein du réseau, lui permettant d'apprendre des frontières de décision complexes et non linéaires nécessaires à la classification d'images.
- **Nécessité absolue :** Sans fonction d'activation non linéaire, l'empilement de plusieurs couches linéaires successives se réduirait mathématiquement à une seule transformation linéaire globale. Le réseau équivaudrait alors à une simple régression linéaire et perdrait toute sa capacité d'apprentissage profond.

2.3 Entraînement supervisé et Fonction de Perte (Entropie Croisée & SGD)

L'entraînement du MLP s'effectue de manière supervisée en ajustant itérativement les poids et les biais du réseau afin de réduire la marge d'erreur :

- **Fonction de perte (Entropie Croisée) :** Elle mesure l'écart entre les prédictions fournies par le réseau et les véritables étiquettes des images. Elle est particulièrement adaptée aux problèmes de classification multiclasse.
- **Calcul des gradients (Autograd) :** La rétropropagation calcule automatiquement les dérivés partielles de la perte par rapport à chaque paramètre du modèle.
- **Optimisation (SGD) :** La descente de gradient stochastique met à jour l'ensemble des poids et biais en soustrayant le gradient multiplié par le taux d'apprentissage (learning rate).

2.4 Concepts de Capacité, Sous-apprentissage et Surapprentissage

Le nombre de neurones sur la couche cachée (num_hiddens) détermine la capacité globale de représentation du réseau de neurones :

- **Sous-apprentissage (Underfitting) :** Survient lorsque le nombre de neurones cachés est trop faible. Le modèle manque de capacité expressive et n'arrive pas à capturer la structure complexe des données, entraînant de faibles performances.
- **Surapprentissage (Overfitting) :** Survient lorsque la capacité du modèle est excessive ou que le nombre d'époques est trop élevé. Le réseau mémorise les détails spécifiques et le bruit de l'ensemble d'entraînement, ce qui se traduit par une excellente précision en entraînement mais une mauvaise généralisation sur les données de test.
- **Configuration équilibrée :** Une capacité bien dimensionnée permet au modèle de capturer les motifs essentiels des données tout en maintenant une excellente précision de généralisation sur de nouvelles images.

3. Implémentation From Scratch

3.1 Déclaration Manuelle des Paramètres (W1, b1, W2, b2) et Rôle de nn.

Dans cette approche de bas niveau, le réseau de neurones est construit manuellement sans recourir aux modules d'architecture séquentielle de haut niveau de PyTorch.

- **Déclaration des tenseurs :** Les matrices de poids (W_1, W_2) et les vecteurs de biais (b_1, b_2) sont initialisés explicitement sous forme de tenseurs numériques.
- **Rôle de nn.Parameter :** L'enveloppement de ces tenseurs avec la classe nn.Parameter est indispensable. Cela permet d'enregistrer explicitement ces variables dans la liste des

paramètres entraînables du modèle (model.parameters()). Grâce à cela, le module d'autodifférenciation (*autograd*) peut calculer leurs gradients et l'optimiseur peut les mettre à jour efficacement lors de la rétropropagation.

3.2 Écriture Explicite de la Transformation Vectorielle et de la Passe Avant (forward).

La passe avant (*forward pass*) formalise le parcours des données à travers le réseau depuis l'image d'entrée brute jusqu'à l'obtention des scores de prédiction.

- **Nécessité du redimensionnement (Reshape) :** Les images du dataset Fashion-MNIST possèdent une structure spatiale bidimensionnelle de 28 * 28 pixels. Les couches linéaires entièrement connectées nécessitant un format d'entrée unidimensionnel, la fonction reshape((-1, 784)) est appliquée pour aplatir chaque image en un vecteur de 784 caractéristiques.
- **Calculs séquentiels des couches :**
    - La couche cachée calcule la combinaison linéaire des entrées par la matrice W_1 et le biais b_1, puis applique immédiatement la fonction d'activation non linéaire ReLU.
    - La couche de sortie applique la transformation finale à l'aide de la matrice W_2 et du biais b_2 pour produire les logits finaux associés aux 10 classes.

3.3 Analyse de l'Initialisation des Poids et Gestion des Formes des Tenseurs.

La maîtrise de l'initialisation des poids et la vérification rigoureuse des dimensions des tenseurs sont des conditions critiques pour assurer la stabilité de l'apprentissage.

- **Initialisation des poids :** Les poids sont initialisés suivant une distribution gaussienne avec un faible écart-type (scale = 0.01). Une initialisation incorrecte (comme des valeurs trop élevées ou des zéros partout) provoquerait une saturation immédiate des neurones ou un blocage total de la descente de gradient.
- **Vérification des dimensions :**
    - Tenseur d'entrée vectorisé : [Batch_Size, 784] (ex: [256, 784]).
    - Matrice de la couche cachée (W_1) : [784, h] (ex: [784, 256]).
    - Matrice de la couche de sortie (W_2) : [h, 10] (ex: [256, 10]).

4. Implémentation Concise

4.1 Construction de l'Architecture avec les Abstractions Haut Niveau (nn.Sequential, nn.Flatten, nn.Linear, nn.ReLU)

Contrairement à l'approche manuelle (*from scratch*), l'implémentation concise s'appuie sur les modules de haut niveau fournis par la bibliothèque PyTorch (torch.nn). Cette approche simplifie considérablement l'écriture du code, améliore sa lisibilité et réduit les risques d'erreurs d'implémentation.

- **nn.Sequential :** Conteneur séquentiel qui enchaîne les différentes couches du réseau dans l'ordre de leur déclaration, permettant un passage fluide des données de la couche d'entrée vers la sortie.
- **nn.Flatten :** Couche spécialisée qui automatise le redimensionnement des tenseurs, transformant instantanément les images 2D de taille 28 * 28 pixels en un vecteur plat de 784 caractéristiques sans nécessiter de manipulation manuelle.
- **nn.Linear :** Couche entièrement connectée (*fully connected*) qui gère en interne la création, l'initialisation et la gestion des matrices de poids et des biais.
- **nn.ReLU :** Fonction d'activation non linéaire insérée entre les couches pour apporter de la complexité aux représentations apprises.

```
ConciseMLP(
  (net): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=256, bias=True)
    (2): ReLU()
    (3): Linear(in_features=256, out_features=10, bias=True)
  )
)
Nombre total de paramètres : 203530
```

Figure 1:Structure séquentielle et nombre total de paramètres du modèle concis

4.2 Tableau Comparatif : Implémentation From Scratch vs Implémentation Concise

Critères de Comparaison	Implémentation From Scratch	Implémentation Concise
Lignes de code	Plus long et verbeux (gestion manuelle des tenseurs et du redimensionnement)	Court, compact et structuré (utilisation de modules prêts à l'emploi)
Gestion des paramètres	Manuelle (déclaration explicite avec nn.Parameter et initialisation des poids)	Automatique (les modules nn.Linear gèrent poids et biais en interne)
Lisibilité et Maintenance	Plus complexe à maintenir et sujette aux erreurs de dimensions	Excellente lisibilité, code modulaire et standardisé
Flexibilité de bas niveau	Contrôle total sur chaque opération mathématique et gradient	Abstraction de haut niveau, idéale pour concevoir des architectures complexes rapidement

Tableau 1:Tableau Comparatif

4.3 Inspection Interne du Modèle (Dimensions, Poids, Biais et Mécanisme d'Autograd)

L'inspection de la structure interne du modèle permet de s'assurer de la conformité des architectures avant l'entraînement.

- **Inspection des paramètres :** Il est possible d'accéder à tout moment aux poids et aux biais de chaque couche via les attributs du modèle (par exemple, model.named_parameters()). Cela permet de vérifier que les dimensions des matrices correspondent bien aux attentes (ex: poids de la couche cachée de taille [256, 784] pour 256 neurones cachés).
- **Vérification des formes (*shapes*) :** L'affichage des dimensions des tenseurs à la sortie de chaque bloc confirme la bonne transition des données à travers le réseau séquentiel.
- **Mécanisme d'Autograd :** Même avec l'approche concise, les modules intégrés de PyTorch activent automatiquement le suivi des gradients (requires_grad=True) sur

l'ensemble des poids, garantissant un fonctionnement transparent de la rétropropagation lors de la phase d'optimisation.

```
Shape des images d'entrée (batch) : torch.Size([256, 1, 28, 28])
Shape après aplatissement (Flatten) : torch.Size([256, 784])
Shape de la sortie du modèle (logits) : torch.Size([256, 10])

--- Inspection des poids et biais ---
Shape des poids de la 1ère couche linéaire : torch.Size([256, 784])
Exemple de poids (w1) :
 tensor([[-0.0292,  0.0196, -0.0210, -0.0244,  0.0148],
        [ 0.0196, -0.0047,  0.0145, -0.0055, -0.0343]],
       grad_fn=<SliceBackward0>)
Exemple de biais (b1) :
 tensor([ 0.0012, -0.0130, -0.0309, -0.0254, -0.0028], grad_fn=<SliceBackward0>)

--- Inspection des gradients après backward() ---
Gradient des poids de la 1ère couche :
 tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]])
```

Figure 2:Validation des dimensions des tenseurs et inspection des poids, biais et gradients.

5. Protocole Expérimental

5.1 Environnement d'Exécution et Boucle d'Entraînement Générique

L'ensemble des expérimentations a été réalisé dans un environnement de développement local sous Windows, en utilisant la bibliothèque PyTorch pour la conception et l'entraînement des réseaux de neurones.

- **Environnement technique :** Python, PyTorch, bibliothèques de manipulation de données (NumPy) et de visualisation (Matplotlib).
- **Boucle d'entraînement générique :**
    - L'entraînement s'effectue de manière itérative sur un nombre fixe d'époques.
    - Pour chaque époque, le jeu de données d'entraînement est parcouru par lots (*batches*) successifs.
    - La boucle d'optimisation réalise les étapes fondamentales suivantes : passage avant (*forward*), calcul de la fonction de perte (entropie croisée), réinitialisation des gradients (optimizer.zero_grad()), rétropropagation (loss.backward()), et mise à jour des poids via l'optimiseur (optimizer.step()).
    - À la fin de chaque époque, une phase d'évaluation sur le jeu de test est exécutée pour mesurer la généralisation du modèle hors échantillon.

5.2 Variation de la Taille de la Couche Cachée

Afin d'analyser l'impact de la capacité du réseau sur ses performances, une étude comparative a été menée en faisant varier le nombre de neurones cachés (num_hiddens).

- **Valeurs testées :** L'évaluation a comparé quatre configurations de la couche cachée : 64, 128, 256 et 512 neurones.
- **Objectif de l'analyse :** Observer comment l'augmentation de la complexité du modèle influence la vitesse de convergence, la précision d'entraînement et la précision de test.
- **Constatations :** Une capacité trop faible (ex: 64 neurones) limite l'expressivité du modèle, tandis qu'une capacité trop élevée (ex: 512 neurones) peut introduire des instabilités ou une sensibilité accrue au bruit, la configuration optimale se situant généralement autour de 256 neurones cachés.

5.3 Variation du Taux d'Apprentissage

Le taux d'apprentissage (*learning rate*, noté lr) est l'hyperparamètre le plus critique pour régir la vitesse et la stabilité de la descente de gradient.

- **Rôle et impact :**
    - Un taux d'apprentissage trop élevé provoque des oscillations importantes de la fonction de perte, voire une divergence totale empêchant l'apprentissage.
    - Un taux d'apprentissage trop faible ralentit excessivement la convergence, nécessitant un nombre d'époques très élevé pour atteindre un minimum satisfaisant.
- **Approche adoptée :** Le choix d'une valeur équilibrée (standardisée à 0.1 ou ajustée via des tests préliminaires) garantit une décroissance régulière et stable de la perte au fil des époques, assurant un apprentissage efficace sans compromettre la stabilité numérique.

6. Résultats et Interprétation

6.1 Analyse de la Convergence du Modèle de Référence (Baseline)

L'analyse des courbes d'apprentissage du modèle de référence permet de valider le bon fonctionnement de la boucle d'optimisation et de la descente de gradient.

- **Évolution de la Perte (Loss) :** On observe une décroissance rapide et continue de la fonction de perte (tant pour l'ensemble d'entraînement que pour l'ensemble de test) au fil des époques. Cette diminution progressive témoigne d'une optimisation stable et d'une réduction efficace de l'erreur du modèle.
- **Évolution de la Précision (Accuracy) :** Corrélativement à la perte, la précision augmente de manière soutenue pour atteindre des niveaux stables autour de 84% à 86%. La proximité entre les courbes d'entraînement et de test indique une bonne capacité de généralisation sans signe critique de surapprentissage.

```
Époque [1/10] - Train Loss: 0.8682, Train Acc: 71.37% | Test Loss: 0.7075, Test Acc: 74.82%
Époque [2/10] - Train Loss: 0.5572, Train Acc: 80.70% | Test Loss: 0.5525, Test Acc: 80.86%
Époque [3/10] - Train Loss: 0.4984, Train Acc: 82.46% | Test Loss: 0.5296, Test Acc: 81.05%
Époque [4/10] - Train Loss: 0.4601, Train Acc: 83.94% | Test Loss: 0.4837, Test Acc: 82.89%
Époque [5/10] - Train Loss: 0.4374, Train Acc: 84.72% | Test Loss: 0.5208, Test Acc: 81.46%
Époque [6/10] - Train Loss: 0.4218, Train Acc: 85.22% | Test Loss: 0.4590, Test Acc: 83.91%
Époque [7/10] - Train Loss: 0.4034, Train Acc: 85.84% | Test Loss: 0.4710, Test Acc: 83.54%
Époque [8/10] - Train Loss: 0.3927, Train Acc: 86.13% | Test Loss: 0.4543, Test Acc: 83.51%
Époque [9/10] - Train Loss: 0.3821, Train Acc: 86.56% | Test Loss: 0.4374, Test Acc: 84.44%
Époque [10/10] - Train Loss: 0.3749, Train Acc: 86.79% | Test Loss: 0.4261, Test Acc: 84.44%
```

Figure 3:Suivi textuel de l'entraînement et des performances par époque (Train/Test Loss & Accuracy).

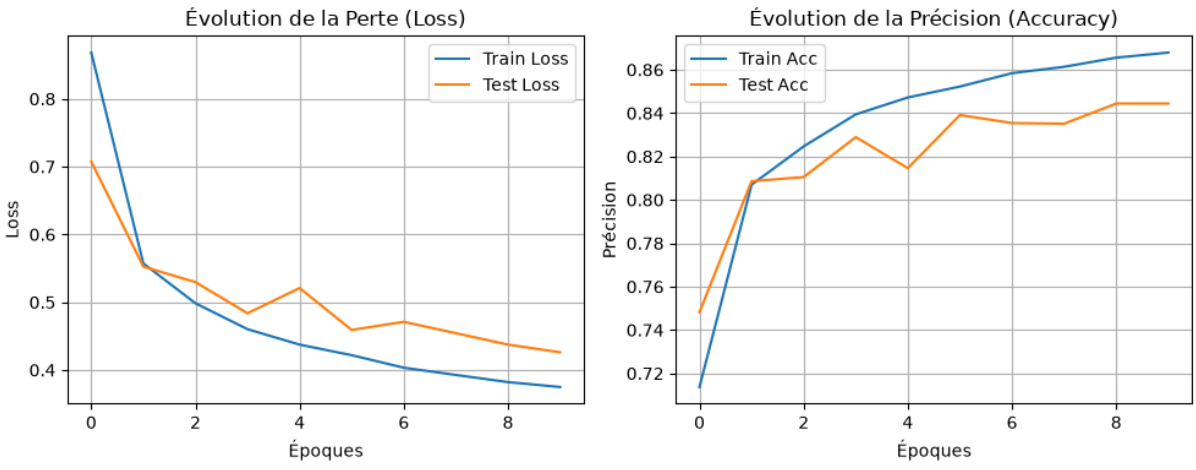


Figure 4:Évolution de la Perte (Loss) et de la Précision (Accuracy) du modèle de référence au fil des époques.

6.2 Analyse Comparative de l'Impact de num_hiddens sur la Précision de Test

L'évaluation comparative menée avec différentes tailles de couches cachées met en lumière l'influence directe de la capacité du réseau sur ses performances finales.

- **Tendances observées :** Les configurations dotées d'un nombre restreint de neurones (ex: 64) affichent une progression plus lente et plafonnent à des niveaux de précision inférieurs. À l'inverse, l'augmentation du nombre de neurones à 256 et 512 permet d'accélérer l'apprentissage des motifs complexes et d'atteindre des pics de performance plus élevés.
- **Stabilité et fluctuations :** Les courbes mettent également en évidence que des capacités trop élevées peuvent introduire des variations plus marquées dans les premières époques avant de se stabiliser, soulignant la nécessité d'un juste équilibre architectural.

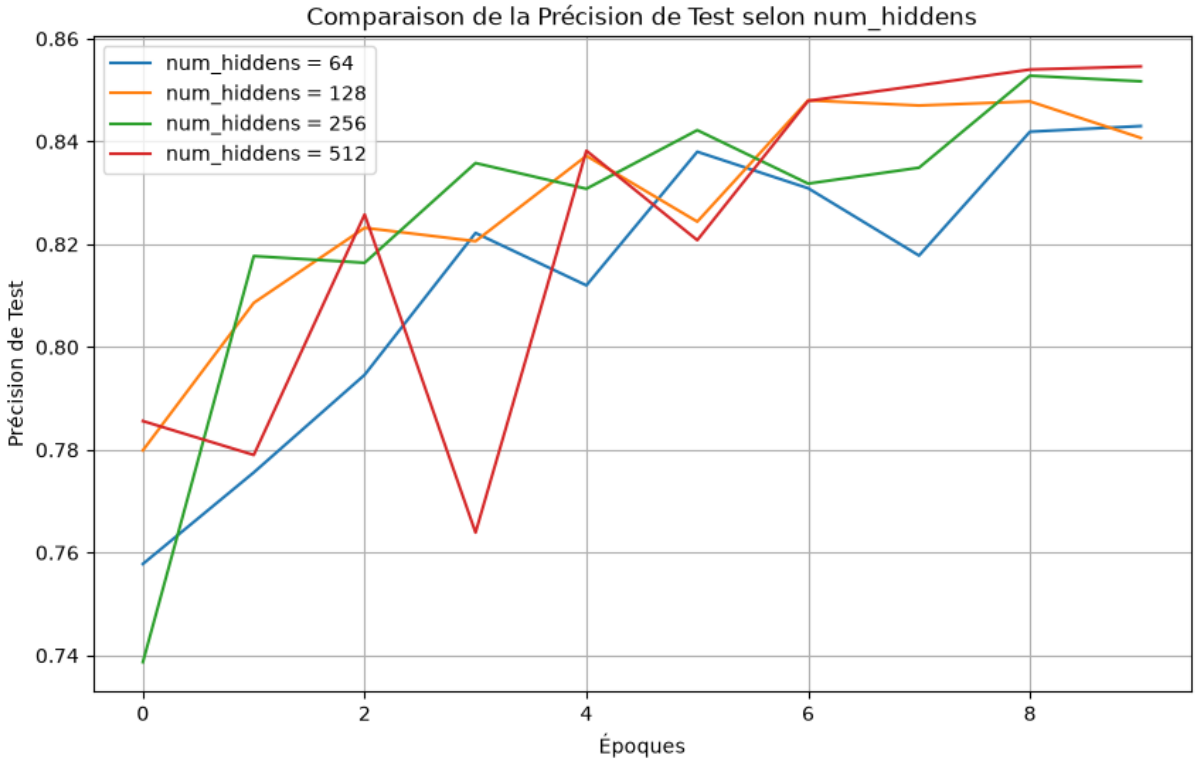


Figure 5:Comparaison de la Précision de Test en fonction du nombre de neurones cachés

6.3 Diagnostic et Identification Expérimentale du Sous-apprentissage et Surapprentissage

L'expérimentation a permis d'observer concrètement les deux grands écueils de l'apprentissage automatique :

- **Sous-apprentissage (*Underfitting*) :** Visible avec les configurations de plus faible capacité ou lors des premières époques. Le modèle ne dispose pas d'assez de degrés de liberté pour modéliser finement la distribution des images Fashion-MNIST, ce qui limite les performances globales.
- **Surapprentissage (*Overfitting*) :** Identifié lorsque le modèle développe une complexité excessive ou s'entraîne de manière prolongée. On remarque alors un écart qui commence à se creuser entre la performance sur les données d'entraînement et celle sur les données de test, le réseau tendant à mémoriser les spécificités du train plutôt qu'à généraliser.

6.4 Sélection et Justification de la Configuration Optimale

À l'issue de l'analyse comparative de l'ensemble des hyperparamètres testés, la configuration optimale retenue est un Perceptron Multicouche doté de **num_hiddens = 256**.

- **Justification :**
    - Elle offre le meilleur compromis entre la puissance d'expression (capacité à capturer la complexité des 10 classes de vêtements) et la stabilité de la généralisation.
    - Elle évite les lourdeurs computationnelles inutiles associées aux plus grands modèles tout en surpassant nettement les performances des configurations plus restreintes (64 ou 128 neurones).

7. Analyse Critique et Limites

7.1 Limites Fondamentales du MLP en Vision par Ordinateur (Perte de la Structure Spatiale 2D et Absence d'Invariance)

Bien que le Perceptron Multicouche (MLP) permette de résoudre des tâches de classification basiques, il présente des limites structurelles majeures lorsqu'il est appliqué au traitement d'images :

- **Destruction de la structure spatiale 2D :** L'aplatissement (*reshape*) indispensable des images en un vecteur unidimensionnel supprime la disposition spatiale des pixels. Le réseau perd ainsi la notion de voisinage direct entre les pixels adjacents, pourtant essentielle pour comprendre les contours et les formes.
- **Absence d'invariance par translation :** Les poids d'un MLP sont fixes et liés à des positions spécifiques de l'image vectorisée. Si un objet se déplace (translation), le modèle doit réapprendre le motif à un autre endroit, car il ne possède pas de mécanisme inné pour reconnaître une caractéristique indépendamment de sa position.

7.2 Problématique de l'Explosion du Nombre de Paramètres sur Images Haute Définition

L'architecture entièrement connectée (*fully connected*) du MLP souffre d'un problème critique de passage à l'échelle :

- **Augmentation exponentielle des poids :** Sur une image de taille modeste (28 * 28 pixels), le nombre de connexions d'entrée est déjà de 784. Pour une image couleur moderne en haute définition (ex: 1920 * 1080 pixels en RVB), la dimension d'entrée dépasse les 6 millions de caractéristiques.
- **Conséquences opérationnelles :** Couplée à une couche cachée de taille raisonnable, cette dimension entraîne une explosion du nombre de paramètres entraînables, provoquant une saturation de la mémoire vive (GPU/CPU), un temps d'entraînement prohibitif et un risque immédiat de surapprentissage (*overfitting*).

7.3 Pertinence et Comparaison avec les Réseaux Convolutifs (CNN)

Face aux limites structurelles du MLP, les Réseaux de Neurones Convolutifs (*Convolutional Neural Networks* ou CNN) s'imposent comme la norme incontournable en vision par ordinateur :

- **Conservation de la topologie spatiale :** Les CNN traitent les images en préservant leur format bidimensionnel ou tridimensionnel d'origine grâce à des filtres locaux.
- **Partage des poids (*Weight Sharing*) :** Contrairement au MLP où chaque neurone possède ses propres poids indépendants, un même filtre convolutif glisse sur toute l'image, réduisant drastiquement le nombre de paramètres à optimiser.
- **Invariance et robustesse :** Grâce aux opérations de convolution et de réduction d'échantillonnage (*pooling*), les CNN intègrent une invariance naturelle aux translations et aux variations deéchelle, surpassant largement les performances des MLP sur des tâches visuelles complexes.

8. Conclusion et Perspectives

8.1 Synthèse des Apports et des Résultats Obtenus

Ce travail a permis de mener à bien la conception, l'implémentation et l'évaluation rigoureuse d'un Perceptron Multicouche (MLP) appliqué à la classification d'images sur le jeu de données Fashion-MNIST.

- **Maîtrise des niveaux d'abstraction :** L'approche s'est articulée autour d'une double implémentation, d'une part *from scratch* pour décortiquer les mécanismes mathématiques de bas niveau et la gestion manuelle des tenseurs, et d'autre part *concise* via les modules de haut niveau de PyTorch pour structurer un code modulaire et robuste.
- **Bilan des performances :** L'étude expérimentale a validé l'efficacité du modèle de référence (*baseline*) et mis en évidence qu'une configuration optimale de la couche cachée (num_hiddens = 256) permet d'atteindre un excellent compromis, stabilisant la précision de test autour de 84% à 86%.

8.2 Réponses Explicites aux Questions de Synthèse

L'étude a permis de répondre concrètement à la problématique initiale posée au cœur de ce projet :

- **Impact de la capacité :** Il a été démontré qu'un nombre insuffisant de neurones (num_hiddens = 64) conduit à un sous-apprentissage (*underfitting*), tandis qu'une capacité excessive ou un entraînement prolongé expose le réseau aux risques de surapprentissage (*overfitting*).
- **Rôle des hyperparamètres :** La maîtrise conjointe du taux d'apprentissage (*learning rate*) et de la taille du réseau s'avère indispensable pour garantir une convergence stable et fluide de la descente de gradient stochastique (SGD).

8.3 Perspectives d'Amélioration (Early Stopping, Dropout, Régularisation L2)

Afin d'aller plus loin et d'optimiser davantage les performances et la robustesse du modèle face à la généralisation, plusieurs pistes d'amélioration majeure peuvent être envisagées :

- **Early Stopping (Arrêt Précoce) :** Mise en place d'un mécanisme de surveillance de la perte sur le jeu de validation pour stopper l'entraînement dès que les performances cessent de s'améliorer, évitant ainsi tout surapprentissage inutile.
- **Dropout :** Introduction de couches de *dropout* pour désactiver aléatoirement un pourcentage de neurones à chaque époque, forçant le réseau à développer des représentations distribuées plus robustes.
- **Régularisation L2 (Weight Decay) :** Application d'une pénalité sur la magnitude des poids au sein de l'optimiseur pour limiter la complexité superflue du modèle.
- **Migration vers les CNN :** Évolution naturelle vers les Réseaux de Neurones Convolutifs (*Convolutional Neural Networks*) afin de s'affranchir des limites du MLP, en préservant la structure spatiale bidimensionnelle des images et en exploitant le partage des poids.